AutoEdge: Towards Full-Lifecycle Agentic Deployment of Edge Video Analytics

Jingzong Li, *Member, IEEE*, Caoxi Zhang, Huanqi Yang, Hai Liu, Hong Xu, *Senior Member, IEEE*, and Guoliang Xing, *Fellow, IEEE*

***Abstract*—The proliferation of video analytics applications on edge devices demands efficient and automated deployment solutions. However, developing these pipelines remains labor-intensive and error-prone due to complex trade-offs between model accuracy, inference latency, and hardware constraints. While Large Language Models (LLMs) have demonstrated strong capabilities in code generation, existing LLM-based approaches lack system awareness, failing to account for edge device resource limitations and resulting in suboptimal or infeasible deployments. We present AutoEdge, the first agentic system designed to automate the full-lifecycle deployment of edge video analytics. AutoEdge bridges the gap between high-level user intent and robust, SLO-aware deployment through three key components: an Intelligent Query Planner that translates natural language intent into concrete execution plans using a hardware-aware model selection algorithm; a Multi-Agent Code Generator that coordinates specialized agents to synthesize, validate, and iteratively refine deployment-ready code; and an Adaptive Online Monitor that continuously tracks runtime performance and dynamically adapts the pipeline to sustain SLOs in non-stationary environments. We implement and evaluate AutoEdge on NVIDIA Jetson devices and Raspberry Pi 5, spanning GPU-accelerated and CPU-based edge platforms, using diverse real-world video analytics queries. Experimental results demonstrate that AutoEdge achieves a 41% improvement in deployment success rate compared to existing LLM-based baselines, while strictly meeting latency SLOs.**

***Index Terms*—Video Analytics, LLM Agents, Edge Computing.**

I. Introduction

Recent years have witnessed a proliferation of video analytics applications across diverse domains, ranging from smart city surveillance [1], [2], autonomous traffic management [3], [4], to industrial safety monitoring [5], [6]. Driven by the need for low-latency processing and enhanced privacy, these applications are increasingly migrating to the edge, where complex deep learning pipelines extract real-time insights directly from high-resolution video [2], [7], [8], [9]. This shift is reflected in the market's explosive trajectory, which is projected to reach $33.74 billion by 2030 with edge deployments leading the growth at 26.3% [10].

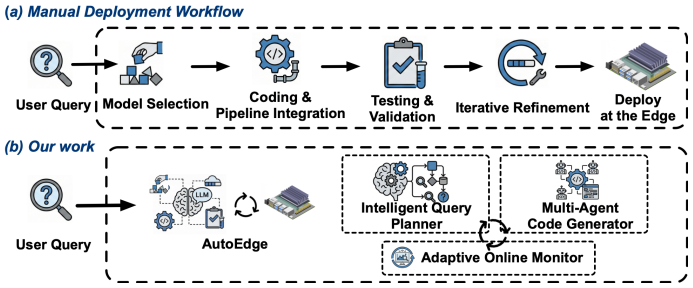


Fig. 1: (a) Existing time-consuming manual workflow of the edge vision deployment. (b) Fully automated workflow with AutoEdge.

Despite the promise of edge video analytics, deploying these pipelines remains labor-intensive and error-prone. As illustrated in Fig. 1(a), developers must manually navigate large configuration spaces, integrate pipelines, and validate them on heterogeneous edge hardware. Section II-A details this workflow and its associated engineering challenges.

Recent advances in LLMs [11], [12], [13] and coding agents [14], [15], [16] offer a potential path toward automation, but existing approaches primarily support code synthesis or isolated development stages [17], [18]. As shown in Table I, they lack the *system awareness* required for full-lifecycle edge deployment. Here, system awareness refers to the ability to account for hardware constraints, application SLOs, pipeline dependencies, and runtime conditions throughout the deployment lifecycle. Consequently, existing approaches cannot reliably translate user intent into an end-to-end deployment that satisfies device-specific constraints and latency SLOs.

To overcome the issues, we present AutoEdge, an agentic system designed to bridge the gap between high-level user intent and robust, SLO-aware edge vision deployment. As illustrated in Fig. 1(b), AutoEdge orchestrates the full development lifecycle, from hardware-aware model selection, programming, and testing to deployment and monitoring, thereby automating the creation of optimized edge video analytics pipelines. This requires a fundamental rethink of how LLMs interact with systems: rather than treating the LLM as a simple code completion engine, we must integrate it into a closed-loop control system that spans planning, implementation, and runtime adaptation.

To realize the system, four core challenges need to be addressed. First, the ambiguity of natural-language user intent

This work was supported in part by funding from HSUHK (SDSC-SRG066, SDSC-SRG087).

Jingzong Li, Caoxi Zhang, and Hai Liu are with the Department of Computer Science, The Hang Seng University of Hong Kong, Hong Kong SAR, China (Email: jingzongli@hsu.edu.hk; caoxizhang@hsu.edu.hk; hliu@hsu.edu.hk).

Huanqi Yang is with the Department of Computer Science, City University of Hong Kong, Hong Kong SAR, China (Email: huanqi.yang@my.cityu.edu.hk).

Hong Xu is with the Department of Computer Science and Engineering, The Chinese University of Hong Kong, Hong Kong SAR, China (Email: hongxu@cuhk.edu.hk).

Guoliang Xing is with the Department of Information Engineering, The Chinese University of Hong Kong, Hong Kong SAR, China (Email: glxing@cuhk.edu.hk).

makes it difficult to reliably extract important information. Second, model selection must account for heterogeneous resource constraints and their intertwined effects on latency to satisfy SLOs. Third, LLM hallucination can lead to incorrect model choices or processing pipelines that fail to preserve user intent. Finally, post-deployment runtime variability, including workload changes and resource contention, can cause a previously feasible pipeline to violate its latency SLO.

AutoEdge addresses the above challenges through three key design components: (i) Intelligent Query Planner: This component maps ambiguous natural language queries into concrete execution plans. To overcome the lack of hardware awareness in standard LLMs, the planner leverages a hardware-specific knowledge base and algorithm to select model architectures and parameters that are strictly grounded in the resource constraints and computational capabilities of the target edge device. (ii) Multi-Agent Code Generator: To eliminate hallucination and ensure functional reliability, we employ collaborative multi-agent orchestration. By coordinating specialized agents, acting as developers and validators, the system iteratively synthesizes, validates, and refines the pipeline code before deployment. (iii) Adaptive Online Monitor: Recognizing that edge environments are inherently dynamic, this component operates post-deployment to ensure sustained compliance with application SLOs. It continuously tracks latency variations and triggers proactive model updates to maintain performance stability.

We have implemented and evaluated AutoEdge on NVIDIA Jetson edge devices, covering various computational capabilities and hardware constraints. Our evaluation across diverse real-world user queries demonstrates that AutoEdge achieves a superior end-to-end success rate of 96%, surpassing the best baseline Cursor by 41%. Notably, AutoEdge consistently achieves perfect coding accuracy across all query types, outperforming existing LLM-based methods which struggle with complex, multi-stage pipelines. For instance, on license plate recognition tasks, AutoEdge maintains a success rate of 86%, while baseline methods degrade to as low as 0% due to compounding errors in model selection and code generation.

We summarize our key contributions as follows:

- We design and build AutoEdge, the first agentic framework to demonstrate systems-aware, full-lifecycle automation of edge video analytics deployment, from user intent to real deployment.
- We develop three mechanisms that make this automation reliable: profile-grounded model selection that jointly considers hardware resources and latency constraints, multi-agent code generation with failure localization and closed-loop validation, and deterministic online adaptation using runtime latency feedback.
- We conduct extensive experiments on NVIDIA Jetson devices and Raspberry Pi 5, covering GPU-accelerated and CPU-based edge platforms. The results show that AutoEdge significantly reduces deployment overhead and outperforms existing LLM-based approaches by 41% in success rate.

Type	Name	Understand Query	Hardware-aware Model selection	Tool Integra.	Auto Debug &Deploy	Monitor
LLMs	GPT-5 [11]	◗	✘	✘	✘	✘
	DeepSeek [13]	◗	✘	✘	✘	✘
	Gemini [12]	◗	✘	✘	✘	✘
Coding Agents	Cursor [14]	◗	✘	✔	✘	✘
	Copilot [15]	◗	✘	✔	✘	✘
	Devin [16]	◗	✘	✔	✘	✘
AutoEdge (Ours)		✔	✔	✔	✔	✔

TABLE I: AutoEdge vs LLMs vs Coding Agents. ◗ means partial support.

II. Background and Motivation

A. Video Analytics at the Edge.

Edge video analytics processes user queries by performing Deep Neural Network (DNN) inference on incoming frames directly at the edge, ensuring low latency and enhanced privacy [1], [2], [7]. They typically run a set of canonical computer vision workloads, each with distinct computational footprints and inference characteristics. In this paper, we focus on six representative query types that have been prevalent in recent literature [19], [20], [21] and real-world deployments [22], [23], [24]. The pipeline for each query type is illustrated in Fig. 2 and described below:

- Object Detection (OD): identifies and localizes all the objects of interest in the frames. It returns the exact bounding boxes of objects and only involves one task.
- Binary Classification (BC): asks whether an object of interest is present or absent in a frame. It returns a Boolean result and involves only one task.
- Counting (CT): returns the number of objects of a specified type in each frame. It uses a single DNN model and shares the same model as object detection.
- Tracking (TR): maintains consistent identities across frames, enabling systems to understand trajectories of objects. It involves two tasks: an object detection model to localize objects, and a separate tracking model to associate those objects across consecutive frames.
- Helmet Violation Detection (HVD): identifies motorcyclists who are not wearing required helmets through two tasks: detection of all motorcyclists, followed by helmet classification.
- License Plate Recognition (LPR): extracts and recognizes license plate text from vehicles in each frame. It involves three tasks: vehicle detection, license plate detection, and optical character recognition (OCR).

Manual Development Workflow. Despite the importance of edge video analytics, today's development pipelines remain largely handcrafted and time-consuming. As illustrated in Fig. 1, users first issue a query that specifies the task and latency SLOs based on application requirements. After receiving the query, the workflow typically proceeds in several steps. 1) First, engineers interpret the query and determine the appropriate model types based on their experience. They then select a suitable model from a pool of candidates with different architectures and sizes, while considering hardware constraints such as GPU, CPU, memory, input resolution, and latency requirements. 2) Next, engineers implement the video

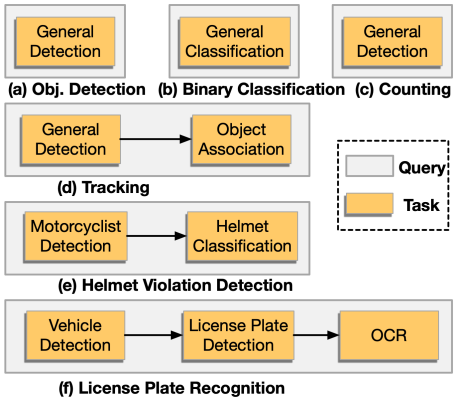


Fig. 2: Pipelines of representative vision queries. Different queries involve varying numbers of sequentially executed tasks.

analytics pipeline, including pre-processing, model inference, and post-processing. 3) After coding, the pipeline must then be tested and validated directly on the target device, since real-world performance depends heavily on hardware characteristics and system-level interactions. 4) If the pipeline fails to run or meet the latency SLO, engineers often rely on iterative trial and error, such as switching to different model variants, reintegrating the pipeline, and revalidating the system. 5) After several iterations, a suitable configuration is eventually deployed on the target edge device.

Based on our empirical observations, even expert developers require around 3–6 hours to successfully deploy an edge vision application with appropriate models, not including the additional effort needed for post-deployment monitoring. Large deployments with diverse devices only amplify this complexity.

Existing Pain Points. This manual workflow exposes several fundamental pain points. First, *the heterogeneity of edge hardware* makes it hard to predict the model performance (as described in §II-C): a model that performs well on one device may run slowly or fail on another due to differences in accelerator capabilities, memory bandwidth, or other hardware constraints. Thus, engineers must rely on experience to select models, often leading to suboptimal performance on the target device. Second, *engineering cost* remains high, as engineers must manually iterate through trial-and-error processes to identify the best models, and any workload change or pipeline modification triggers another cycle of redeployment and validation. At scale, large deployments with diverse devices only amplify this complexity, not to mention the high cost of hiring experienced engineers. Lastly, the existing workflow *lacks automated validation* and is thus error-prone: since the manual process depends heavily on expert knowledge, there is no standard mechanism to verify correctness, stability, or latency across large, distributed fleets of edge nodes.

B. LLM-based AI Agents.

Large Language Models (LLMs) have become the dominant foundation for natural language understanding and generation. Built on Transformer architectures [25] and trained on massive text corpora [26], [27], they scale to billions of parameters and exhibit strong capabilities in multi-hop reasoning [28], code synthesis [29], and domain adaptation [30]. Representative examples include the GPT-4/5 series [11], DeepSeek [13], and Gemini [12]. Building on these advances, recent work has shifted from using LLMs as standalone response generators to integrating them as the core of AI agents. LLM-based agents leverage LLMs to plan, act, observe, and iteratively refine their behavior through closed-loop interaction with the environment [31], [32], enabling more complex tasks such as workflow automation and long-horizon problem solving. Representative examples include GitHub Copilot [15], Cursor [14], and Devin [16], which are coding agents designed to autonomously tackle tasks such as writing, executing, and debugging code.

Opportunity: These advances motivate a key research question: *Can LLMs be harnessed to build an autonomous agentic system, to address the long-standing pains of edge video analytics deployment?*

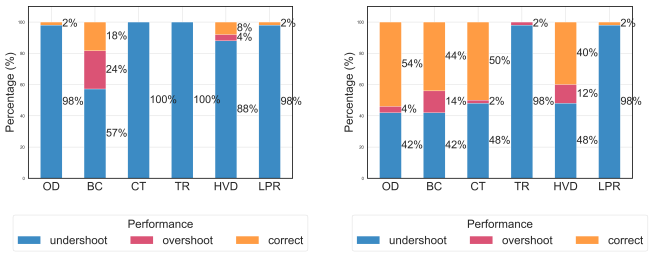
C. Challenges

To achieve the above goal, four fundamental challenges need to be addressed in practice. We describe them in turn.

C1: Ambiguity in Natural-Language User Intent. Users may express the same analytics goal in diverse or underspecified ways, leaving essential fields such as the query type, task sequence, camera source, or latency SLO implicit or ambiguous. Before deployment can begin, an agent must reliably translate this open-ended request into a complete and structured representation while identifying missing or inconsistent requirements. Errors at this stage propagate to model selection and code generation, causing the resulting pipeline to deviate from the user's intent.

C2: Hardware/SLO-Aware Model Selection. Edge devices are typically built around system-on-chip (SoC) architectures that integrate CPUs, GPUs, and memory to operate under tight power and latency constraints [33], [34], [35], [36]. Compared to desktop/server-class GPUs, edge GPUs offer substantially lower compute capability and smaller memory capacity. Fig. 4 highlights this hardware gap: representative edge devices provide over 10× less compute throughput and 6× less memory than traditional server GPUs. Moreover, edge CPUs and GPUs often share a unified memory pool [37], [38], which further constrains available memory resources. Given the diversity and tight constraints of edge environments, selecting vision models that both fit the available hardware and satisfy stringent latency SLO is challenging even for experienced engineers, let alone for LLMs tasked with making these non-trivial selections.

We conduct a motivational experiment to evaluate the correctness of model selection by the latest GPT-5 [11]. We consider two representative edge devices: Jetson Orin Nano [37] and Orin NX [38]. For each task type, we construct 50 natural-language queries that specify the video analytics task, hardware characteristics, and latency SLO, and ask GPT-5 to select appropriate model(s) from a predefined model list. As shown in Fig.3, GPT-5 frequently selects inappropriate models, either choosing overly large models that violate latency constraints (i.e., overshooting) or overly small models that underutilize available resources (i.e., undershooting).



(a) Jetson Orin Nano (b) Jetson Orin NX

Fig. 3: Correctness of GPT-5's model selection for the Jetson Orin Nano and Orin NX. Across all tasks, GPT-5 frequently overshoots or undershoots the appropriate model choice for the edge devices.

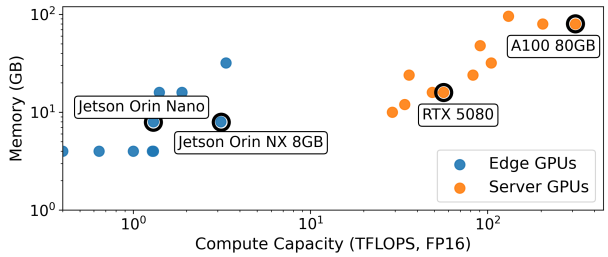


Fig. 4: Resource comparison of edge and server GPUs.

Intertwined Hardware Effects. Selecting vision models is inherently challenging, as it must identify the correct model type from a diverse model zoo based on user intent and simultaneously choose a model size that meets strict latency SLOs while maximizing accuracy. Hardware characteristics of the target device play a critical role in this selection. As shown in Fig. 5, inference latency is influenced by multiple factors, including CPU, GPU, and memory. Moreover, vision models exhibit heterogeneous sensitivity to these attributes. For instance, CPUs have a comparatively limited impact, as the CPU primarily performs lightweight preprocessing and postprocessing, while the GPU executes the highly parallel matrix operations that dominate inference time. Beyond compute resources, memory bandwidth plays a pivotal role in determining inference latency (Fig. 5(e)). Although model weights are loaded once at initialization, they are repeatedly accessed during each forward pass, and intermediate activations are continuously read and written throughout inference. The limited memory bandwidth of edge devices—typically 10–60 GB/s compared to hundreds of GB/s on discrete GPUs [37][39]—makes vision models particularly sensitive to bandwidth constraints.

Input characteristics such as video resolution further compound these effects, as higher-resolution inputs increase both computation and memory traffic, leading to higher inference latency. Together, the complex interplay among hardware constraints, model characteristics, and input resolution makes accurate model selection for edge video analytics non-trivial and error-prone, even for experienced engineers.

C3: Hallucination in LLM's Decision-Making. Hallucination is a widely recognized challenge in all LLMs and remains an active area of research in the AI community [40], [41], [42]. It arises because LLMs generate responses based on learned language patterns, instead of being explicitly grounded in factual knowledge [42]. Compared to tasks such as casual conversation or report writing, the autonomous development of video analytics pipelines demands a higher level of output correctness. As observed in our design, LLMs may misinterpret user intent, for example, by inferring incorrect query types or selecting incompatible model pipelines that do not align with the specified query or hardware constraints. To harness the reasoning capabilities of LLMs while avoiding such failures, it is necessary to design effective mechanisms that detect and mitigate hallucinations during the development process.

C4: Post-Deployment Runtime Variability. Even a pipeline that initially satisfies its latency SLO operates in a non-stationary environment. Changes in video content can alter inference and post-processing workloads, while background processes, resource contention, and thermal throttling can reduce the available CPU/GPU capacity [43], [8], [44], [22]. Consequently, a static model configuration may become either infeasible or unnecessarily conservative after deployment, requiring continuous monitoring and runtime adaptation.

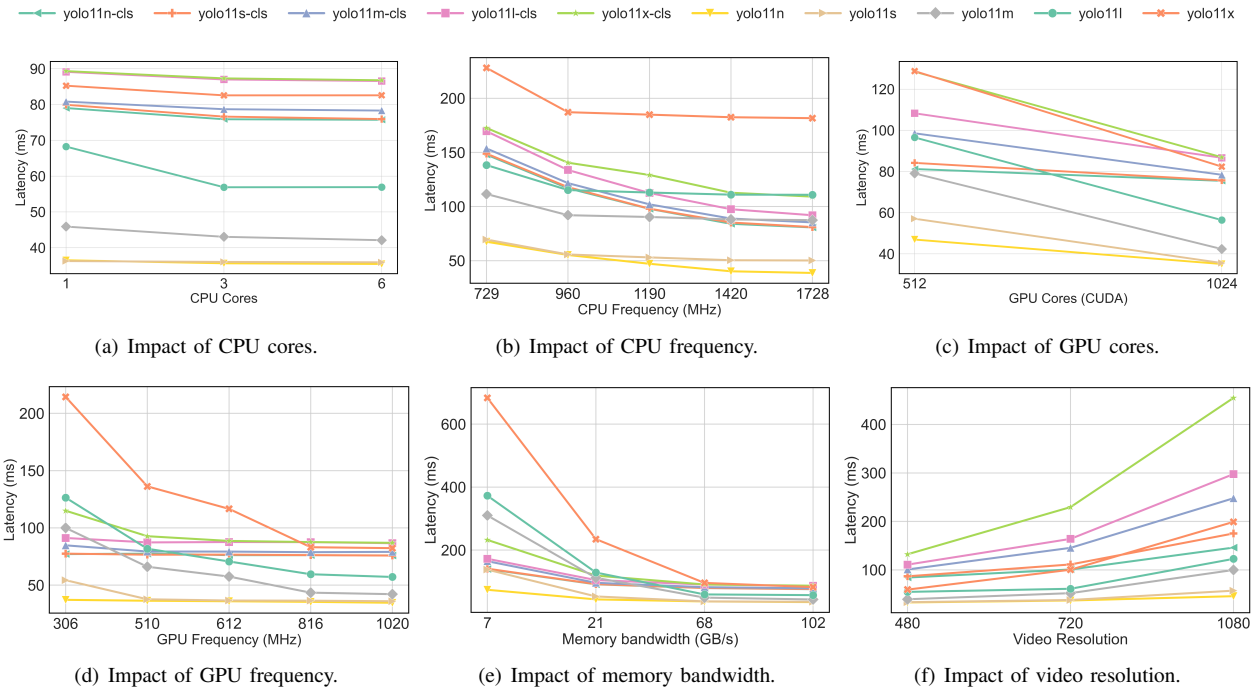
III. System Design

A. System Overview

AutoEdge consists of three core components: (1) *Intelligent Query Planner* (§III-B) interprets user intent and derives a concrete execution plan tailored to the constraints of the target edge device. (2) *Multi-Agent Code Generator* (§III-C) executes this plan by coordinating multiple specialized agents to synthesize, validate, and iteratively refine deployment-ready code. (3) *Adaptive Online Monitor* (§III-D) operates post-deployment, continuously tracking hardware resource variations to ensure sustained compliance with application SLOs.

Fig. 6 shows AutoEdge 's end-to-end workflow. In phase 1, users interact with AutoEdge via natural-language queries that specify the analytics task, objects of interest, and latency SLO. Upon receiving a query, a planner agent interprets user intent and system requirements, and triggers hardware-aware model selection to select appropriate models from the model zoo by jointly considering query needs and hardware constraints. Knowledge preparation is performed offline to support the model selection process. AutoEdge then generates a concrete execution plan specifying the selected model(s) and pipeline. In Phase 2, multiple coding agents collaborate to synthesize code, which is then integrated and validated within a safe, controlled sandbox. If errors arise, a validator agent identifies the root cause and triggers targeted code regeneration, forming a closed validation loop. In Phase 3, AutoEdge deploys the validated pipeline onto the edge device. After deployment, the system continuously monitors runtime performance and dynamically adapts model choices and configurations to maintain compliance with latency SLOs.

Usage Scenario. Consider a traffic monitoring scenario, an operator issues a natural-language query to AutoEdge, requesting vehicle detection on a specific camera stream with an end-to-end latency SLO of 100 ms. AutoEdge interprets the



(a) Impact of CPU cores. (b) Impact of CPU frequency. (c) Impact of GPU cores.

(d) Impact of GPU frequency. (e) Impact of memory bandwidth. (f) Impact of video resolution.

Fig. 5: Impact of hardware attributes and video characteristic on the inference latency of representative vision models on edge devices. Measurements are conducted on the Nvidia Jetson series.

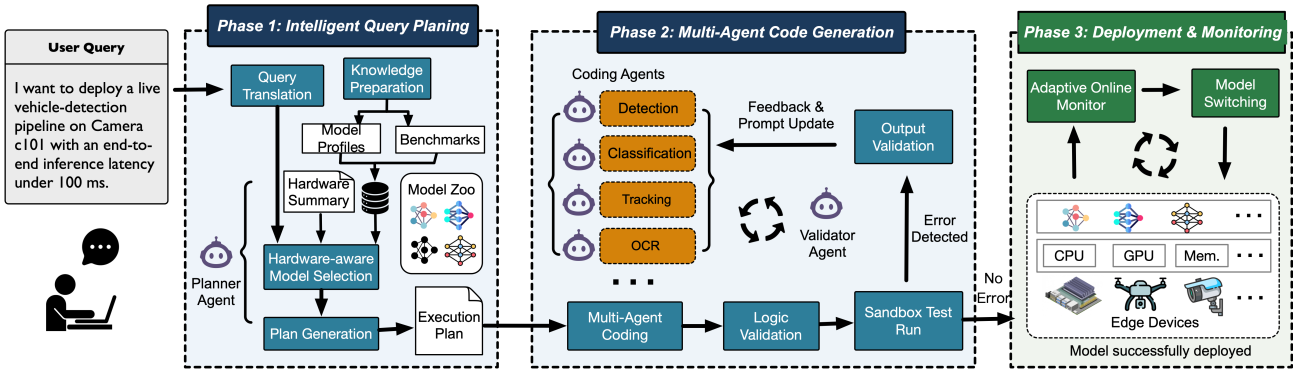


Fig. 6: AutoEdge system overview.

request, profiles the target hardware, and selects a detection model that satisfies the latency constraint while maximizing accuracy. It then automatically generates deployment-ready code and launches the analytics pipeline on the target device. After deployment, AutoEdge continuously monitors runtime performance and dynamically adapts the pipeline, e.g., by switching to a more efficient model variant, to preserve real-time performance. All of these steps are triggered by a single high-level user command, without manual intervention.

B. Intelligent Query Planning

The primary goal of AutoEdge's intelligent query planning is to interpret user queries and select vision models that best fit the target edge device while satisfying strict latency SLOs.

Query Translation. AutoEdge first employs an LLM-based *Planner Agent* to interpret user queries. Given a natural-language query q, the planner agent parses the input and extracts user intent, producing a structured JSON representation that includes the query type q_{type}, task sequence $\mathcal{R} = \{t_1, t_2, \ldots, t_K\}$, camera identifier $\mathcal{C}$, and latency constraint ℓ_{SLO}. Although LLMs are effective at understanding unstructured language, we observe that they can still misinterpret task

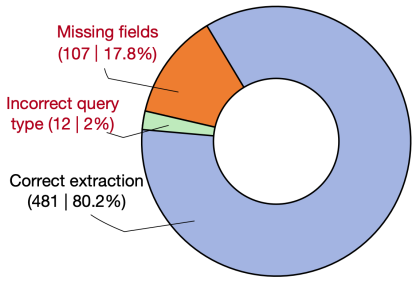


Fig. 7: Breakdown of query translation failures using the original prompt. Errors arise from missing fields and incorrect query type.

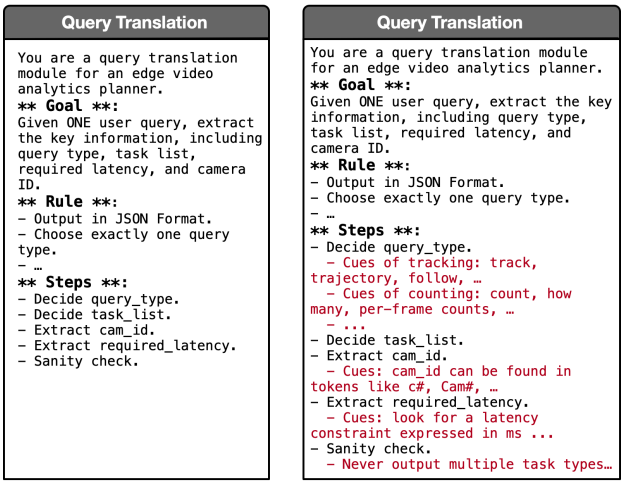


Fig. 8: Failure-guided prompt refinement for query translation.

semantics or miss critical fields due to query diversity and ambiguity. As shown in Fig. 7, 120 out of 600 queries are incorrectly parsed using the basic prompt in Fig. 8(a). Notably, 17.8% of errors stem from missing fields due to incomplete extraction, while a smaller portion arises from incorrect task-type identification. These errors can propagate downstream, leading to infeasible deployments or runtime failures.

Failure-Guided Prompt Refinement. Rather than attempting to exhaustively enumerate all possible query variations, AutoEdge adopts a failure-guided prompt refinement mechanism that incrementally improves query translation based on errors observed during execution. A key insight is that many query-understanding failures manifest as downstream execution errors. For example, a missing camera ID can lead to failures when later retrieving the hardware summary. Because AutoEdge automates the full development pipeline, the system naturally produces a set of failed cases that can be traced back to the earlier query translation stage. We treat such failures as first-class signals. Each failure is logged together with: (i) the original user query, (ii) the structured intent extracted by the planner agent, and (iii) the observed failure reason. This creates a continuously growing corpus grounded in real system behavior. Instead of modifying LLM weights or introducing additional learning components, AutoEdge employs a lightweight strategy called failure-guided prompt refinement to improve the accuracy of query translation. Periodically, a small batch of logged failures is analyzed in an offline setting, where the LLM is prompted to propose targeted improvements to the planner agent's prompt. These refinements may include:

- Adding explicit cues for each reasoning step, or
- Disambiguating task-specific terminology, or
- Enforcing stricter output constraints for critical fields, etc.

All prompt updates are reviewed before being incorporated into the planner agent. This approach grounds prompt refinement in real execution failures while avoiding the complexity and overhead of LLM fine-tuning. Fig. 8(b) presents an example of prompt updates derived from real system executions.

Problem Definition. After the user query has been successfully parsed, AutoEdge needs to select the vision model for each task in the query type. The goal is to maximize overall accuracy while satisfying latency SLOs. We start by defining the problem: Let

$$\mathcal{M} = \bigcup_{t \in \mathcal{T}} \mathcal{M}_t \tag{1}$$

denote a model zoo, where $\mathcal{T}$ is the set of vision task types (e.g., vehicle detection, license plate detection) as illustrated in the yellow blocks of Fig. 2, and $\mathcal{M}_t$ is the set of candidate models for task type t. Each model $m \in \mathcal{M}_t$ is associated with a task type $t(m) \in \mathcal{T}$. We assume that within each task type t, model accuracy is monotonically increasing with model size. That is, for any two models $m_1, m_2 \in \mathcal{M}_t$,

$$s(m) > s(m') \Rightarrow Acc(m) > Acc(m') \tag{2}$$

where $s(m)$ denotes the size of model m (e.g., number of parameters) and $Acc(\cdot)$ denotes task-specific accuracy. This assumption reflects common practice in vision model families and allows model size to serve as a proxy for accuracy [45], [46]. Note that when adding new models to the model zoo, we also strictly follow the rule.

For the query q, it specifies a development request and is converted to a structured description by the query translation module:

$$q = (q_{type}, \mathcal{R}, \mathcal{C}, \ell_{\text{SLO}}) \tag{3}$$

where q_{type} is the query type as described in Section II-A, $\mathcal{R} = \{t_1, t_2, ...t_k\} \subseteq \mathcal{T}$ is the set of required task types in the query (a query may involve one or multiple vision tasks), $\mathcal{C}$ is the camera identifier, and $\ell_{\text{SLO}} \in \mathbb{R}^+$ is the end-to-end latency SLO specified by the user.

Thus, for a given query q, the problem is to select a set of models $\mathcal{S} = \{m_1, m_2, ..., m_k\}$, where each $m \in \mathcal{M}_{t_j}$, corresponds to a required task type t_j, such that two objectives can be achieved: 1) Latency feasibility: The combined deployment of models in $\mathcal{S}$ satisfies the latency SLO, where $\mathbf{h}^*$ is the hardware conditions of the target edge device (e.g., CPU cores, GPU cores, memory bandwidth, etc):

$$\hat{\ell}(\mathcal{S}|\mathbf{h}^*) \leq \ell_{\text{SLO}} \tag{4}$$

2) Accuracy maximization: Among all feasible selections, $\mathcal{S}$ maximizes total overall accuracy:

$$\max_{\mathcal{S}} Acc(\mathcal{S}) \tag{5}$$

The two objectives reflect the assumption that users prefer the most accurate deployment that meets the latency constraints while achieving the highest accuracy.

Hardware-Aware Model Selection. To address the above model selection problem, an intuitive solution is to directly prompt the planner agent to perform reasoning based on the hardware summary, SLOs, and candidate models. This treats the LLM as a black box and expects it to implicitly understand the performance characteristics of target edge devices. However, as demonstrated in Fig. 3, LLM-based model selection fails to reliably predict performance for edge devices and often selects overly conservative models. Moreover, LLMs are inherently probabilistic and sensitive to prompt phrasing, temperature, and model version, which is undesirable

for model selection. To ensure performance guarantees and reproducibility, AutoEdge intentionally avoids using LLMs for direct model selection. Instead, we propose a deterministic, hardware-grounded approach, termed the *hardware-aware model selection algorithm*, and encapsulate it as a skill that can be invoked by the planner agent. The details are presented in Algo. 1.

To ground model selection in empirical data while remaining lightweight, we represent each edge device's hardware conditions as a low-dimensional feature vector:

$$\mathbf{h} = [h_1, h_2, ..., h_d] \tag{6}$$

Each dimension h_j captures a salient hardware attribute, such as CPU/GPU core counts and frequencies, memory bandwidth, and supported video resolutions. These features are selected because they are the dominant factors influencing inference latency (see Fig. 5).

To guide the model selection, knowledge preparation is conducted offline to measure model performance on a representative set of devices. This step produces the knowledge base:

$$\mathcal{D} = \{(\mathbf{h}^{(r)}, m^{(r)}, \ell^{(r)}) | r = 1, \ldots, N\} \tag{7}$$

Each record r corresponds to a single measured execution of model $m^{(r)}$ on hardware configuration $\mathbf{h}^{(r)}$, resulting in latency $\ell^{(r)}$. Importantly, the goal of this knowledge base is not to exhaustively profile all models on all devices, but to serve as a sparse reference for extrapolating performance to unseen hardware.

Given a target device with hardware vector $\mathbf{h}^*$, AutoEdge first identifies the most similar profiled device $\mathbf{h}_n$ from the knowledge base (line 1), using a weighted distance metric. We select the single closest device as the reference because our knowledge base is constructed from a small set of devices that representatively cover the major hardware tiers, ensuring a close match exists for each target device. The distance is defined by:

$$d(\mathbf{h}^*, \mathbf{h}_i) = \|\mathbf{W}(\mathbf{h}^* - \mathbf{h}_i)\|_2 \tag{8}$$

where $\mathbf{W} = \mathrm{diag}(w_1, w_2, \ldots, w_d)$ is a diagonal weighting matrix that encodes the relative importance of hardware attributes. We set the weights based on the sensitivity analysis in §II-C: GPU-related attributes (core count, frequency, and memory bandwidth) that dominate inference latency receive higher weights, while CPU attributes that primarily affect lightweight pre/post-processing receive lower weights.

To estimate the latency of a model m on the target device, we define a Hardware Capacity Function, $c(\mathbf{h}) = \|\mathbf{W}\mathbf{h}\|_1 = \sum_{j=1}^{d} w_j h_j$, which serves as a monotonic proxy for hardware capability. The estimated latency $\hat{\ell}$ is then calculated by:

$$\hat{\ell}(m \mid \mathbf{h}^*, \mathbf{h}_n) = \ell_i \cdot \frac{c(\mathbf{h}_n)}{c(\mathbf{h}^*)} \tag{9}$$

where ℓ_i is the measured latency of m on the most similar hardware $\mathbf{h}_n$. The total end-to-end latency for a pipeline $\mathcal{S}$ is the sum of these individual estimates:

$$\hat{\ell}(\mathcal{S} \mid \mathbf{h}^*, \mathbf{h}_n) = \sum \hat{\ell}(m \mid \mathbf{h}^*, \mathbf{h}_n) \tag{10}$$

Algorithm 1: Hardware-Aware Model Selection

Input :
- Target hardware features $\mathbf{h}^*$;
- Task sequence $\mathcal{R} = \{t_1, t_2, \ldots, t_K\}$;
- Latency SLO ℓ_{SLO};
- Knowledge base $\mathcal{D}$;

Output:
- Selected model pipeline $\mathcal{S}^*$

1 Retrieve the top-1 nearest hardware neighbor: $\mathbf{h}_n \leftarrow \arg\min_{\mathbf{h}_i:(\mathbf{h}_i, m_i, \ell_i) \in \mathcal{D}} d(\mathbf{h}^*, \mathbf{h}_i)$
2 Initialize feasible pipeline set $\mathcal{F} \leftarrow \emptyset$;
3 **for** *all model pipelines* $\mathcal{S} = (m_1, m_2, \ldots, m_K)$ *where* $m_k \in \mathcal{M}_{t_k}$ **do**
4 Estimate adjusted latency $\hat{\ell}(\mathcal{S} \mid \mathbf{h}^*, \mathbf{h}_n)$;
5 **if** $\hat{\ell}(\mathcal{S} \mid \mathbf{h}^*, \mathbf{h}_n) \leq \ell_{SLO}$ **then**
6 Add $\mathcal{S}$ to $\mathcal{F}$;
7 **end if**
8 **end for**
9 Rank pipelines in $\mathcal{F}$ lexicographically by performance tuples $(p_1(m_1), p_2(m_2), \ldots, p_K(m_K))$: $\mathcal{F}_{\text{ranked}} \leftarrow \text{LexRank}(\mathcal{F})$
10 Select the top-1 pipeline: $\mathcal{S}^* \leftarrow \text{Top-1}(\mathcal{F}_{\text{ranked}})$
11 **return** $\mathcal{S}^*$;

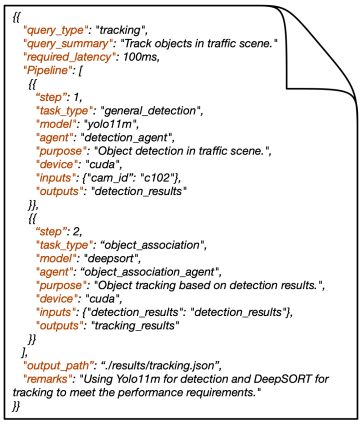
This linear scaling serves as a lightweight approximation that avoids the overhead of per-device profiling. While it does not capture all hardware nonlinearities, our goal is to preserve the correct ranking among candidate pipelines rather than predict absolute latency. As we show in §V-D, the estimation error remains within 2-5 ms under GPU acceleration, which is well below typical inter-model latency gaps. All model pipelines whose estimated latency satisfies the latency constraint are added to the feasible set (line 3-8).

Finally, AutoEdge must select the pipeline that maximizes the accuracy from the feasible set. We adopt a *lexicographic ranking strategy*. This design is motivated by the cascaded nature of video analytics pipelines: errors introduced at earlier stages are propagated downstream and cannot be recovered by later stages. Consequently, overall accuracy highly depends on the early-stage performance. We therefore prioritize maximizing the earliest-stage performance, followed by the second-stage performance, and continue this process sequentially for subsequent stages (line 9). The top-ranked pipeline $\mathcal{S}^*$ is selected as the recommended model set (line 10).

Execution Plan Generation. At the end of the query planning, the planner agent generates a detailed execution plan in a structured JSON format. The plan specifies the parsed query type, model pipeline, corresponding coding agents, and other necessary execution metadata. This execution plan is then passed to the next phase for task execution and validation. An example execution plan is shown in Fig. 9.

C. Multi-Agent Code Generation

Given the execution plan, AutoEdge generates code that faithfully implements the tasks of a query. Our design is driven by two primary objectives: 1) Execution correctness.



```
{{
  "query_type": "tracking",
  "query_summary": "Track objects in traffic scene."
  "required_latency": 100ms,
  "Pipeline": [
   {{
    "step": 1,
    "task_type": "general_detection",
    "model": "yolo11m",
    "agent": "detection_agent",
    "purpose": "Object detection in traffic scene.",
    "device": "cuda",
    "inputs": {"cam_id": "c102"},
    "outputs": "detection_results"
   }},
   {{
    "step": 2,
    "task_type": "object_association",
    "model": "deepsort",
    "agent": "object_association_agent",
    "purpose": "Object tracking based on detection results.",
    "device": "cuda",
    "inputs": {"detection_results": "detection_results"},
    "outputs": "tracking_results"
   }}
  ],
  "output_path": "./results/tracking.json",
  "remarks": "Using Yolo11m for detection and DeepSORT for
tracking to meet the performance requirements."
}}
```

Fig. 9: Example execution plan of the tracking query.

Coding Agent Template

```
You are an Edge AI Implementation
Specialist.
** Goal **
Given a structured JSON execution
plan {task_plan}, generate a self-
contained, production-grade Python
module that implements the target
task.
** Rule **
- Output ONLY Python code.
- Runnable end-to-end.
- Implement only what the plan
specifies.
- Use safe defaults if information is
missing.
- Must include imports, function of
task logic, and main().
** Steps **
- Import necessary libraries:...
- Model initialization:...
- Pre-processing:...
- Inference; include latency
measurement: ...
- Post-processing:...
- Save results to file:...
- Provide executable function call...
```

Fig. 10: Prompt template for coding agents.

```
model = YOLO(f"models/yolo11x-cls.pt")
...
while cap.isOpened():
    ...
    results = model(frame)
    frame_detections = []
    for result in results:
        for box in result.boxes:
            bbox = box.xyxy[0].tolist()
            confidence = box.conf[0].item()
            class_id = int(box.cls[0].item())
            class_name = model.names[class_id]
            frame_detections.append({
                "bbox": bbox,
                "confidence": confidence,
                "class_id": class_id,
                "class_name": class_name
            })
    detections.append(frame_detections)
    total_frames += 1
    ...
```

Fig. 11: Logical hallucination of the single-agent approach. The generated code exhibits "task entanglement", where post-processing logic for object detection is erroneously used for classification tasks.

The generated code should be runtime-safe and executable on the target hardware, and 2) Semantic fidelity: It should fully realize the user's query intent and performance goals.

Single-Agent vs. Multi-Agent. A naive solution for code generation is using a monolithic single agent to handle the implementation of all task types. In this method, the prompts fed into the agent cover the description of all tasks and thus are lengthy. As we have observed in real implementation, the single-agent approach poses two main problems: First, reasoning over multiple tasks within a single prompt often leads to task entanglement, where logic intended for one task inadvertently influences others. As shown in Fig. 11, the LLM-based single agent tangles the logic of different task types, incorrectly applying detection-specific post-processing (the boxes attribute) to a classification result. Second, failures in the generated code are difficult to localize. Because different tasks of a query are intertwined in a single output, identifying the root cause of an error typically requires regenerating the entire code, even when the failure originates from a specific task.

To overcome the problems, AutoEdge adopts a multi-agent design in which each agent is responsible for a single, well-defined task. The illustration is shown in Fig. 12. Specifically, instead of one-time code generation, AutoEdge decomposes the coding process into a sequence of tasks following the execution plan, where each task type is handled by a specialized LLM-based agent (e.g., detection, object association, OCR, etc.). Each agent operates over a bounded context and collaborates with downstream agents via the execution plan. This design follows the single-responsibility principle [47] and mirrors classical modular system design. When a new task type is added, the coding agent can be easily reproduced using the designed prompt template in Fig. 10.

Compared to the single-agent approach, our multi-agent framework introduces several advantages: 1) *Fault Isolation and Selective Regeneration*. Failures in code generation or execution are often localized to a specific task. The multi-agent design allows the system to attribute failures to an individual agent and re-invoke only the faulty component. This contrasts with the monolithic agent, where any failure typically requires re-generating the entire codebase. 2) *Extensibility*. The multi-agent design naturally supports extensibility. New capabilities (e.g., support for new tasks) can be introduced by adding or replacing individual agents without modifying the rest of the system. 3) *Token Efficiency*. In the single-agent approach, prior outputs are repeatedly appended to the prompt, causing token usage to grow quadratically with the number of iterations. In contrast, our design ensures that each agent consumes only task-relevant context, and with selective regeneration, it can achieve linear token growth over retries.

Based on our observations, a single agent is sufficient for a self-contained, single-stage query with bounded context and no cross-model interfaces. Multi-agent coding is preferable for multi-stage queries whose components have distinct interfaces and pre/post-processing logic, or when failures should be localized and repaired independently. Accordingly, AutoEdge uses one coding agent for a single-stage query and activates a sequence of specialized agents only for a multi-stage pipeline.

Automated Code Validation. To ensure the correctness and reliability of the generated code, AutoEdge employs an LLM-based validator agent that serves as a gatekeeper before final deployment. It performs two complementary functions: 1) *Logic validation*: is performed prior to execution and analyzes the generated code to identify semantic errors and unsafe operations that cannot be detected by the interpreter alone. By identifying missing steps, inconsistent workflow, or unsafe system access, the validator provides early feedback to the coding agent. 2) *Output validation*: extracts error signals, including runtime exceptions, and anomalous outputs, and translates them into structured feedback that pinpoints the source of failure. This feedback is fed back to the coding agent to enable targeted correction rather than full regeneration.

Fig. 12 illustrates the end-to-end validation process for a tracking query, handled collaboratively by two coding agents. The first coding agent synthesizes code for object detection (❶), which is immediately forwarded to the validator for logic validation. Here, the validator identifies a missing step—latency is not recorded—and flags it (❷), prompting the coding agent to revise the code accordingly (❸). This logic validation loop repeats until the code is deemed logically correct, after which it is executed in an edge sandbox. Output validation then checks both execution correctness and semantic fidelity (❹); if a runtime exception is caught (e.g., attribute errors), the validator extracts error messages and provides

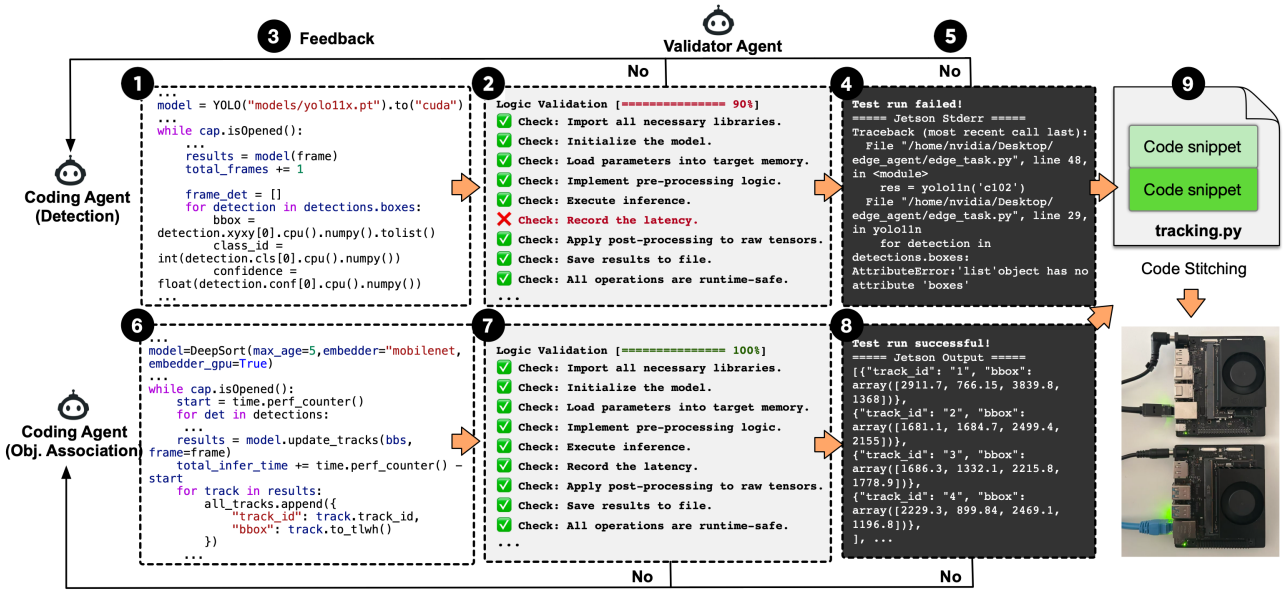


Fig. 12: Example of multi-agent code generation and validation for the tracking query.

feedback to the coding agent for correction (❺). This cycle repeats for subsequent tasks (e.g., object association), with each agent's output refined through the feedback loop until logically and functionally correct (❻–❽). Once all tasks pass both validation stages, the verified code snippets are systematically stitched together and deployed to the edge device (❾).

Edge Sandbox. Since AutoEdge executes LLM-generated code directly on user-owned edge devices, a secure and isolated execution environment is essential. LLM-generated code is inherently unpredictable; even state-of-the-art models can produce code with subtle bugs, infinite loops, or unintended behaviors such as accessing sensitive files. Without proper isolation, such code could compromise the stability of the edge device or leak private data. To address this, AutoEdge employs an *edge sandbox* that confines all code execution within a controlled boundary, ensuring that failures do not propagate to the host system.

We implement the sandbox using Docker containers, which are natively supported on NVIDIA Jetson platforms via JetPack [48]. Docker provides lightweight isolation by leveraging two Linux kernel primitives: *namespaces* and *cgroups*. Namespaces partition kernel resources so that processes inside the container operate with an isolated view of the system. The sandbox cannot inspect host processes, access files outside its designated workspace, or establish unauthorized network connections. Cgroups enforce resource limits; AutoEdge configures memory bounds for the container so that if generated code triggers a memory leak, the kernel terminates the offending process rather than allowing it to exhaust device resources. For GPU access, we use the `--runtime nvidia` flag to invoke the NVIDIA Container Runtime, which exposes CUDA capabilities inside the sandbox while maintaining CPU and memory isolation. The sandbox follows a stateless lifecycle tied to each deployment task. Upon receiving a request, AutoEdge instantiates a fresh container from a base image pre-configured with the video analytics toolchain (Python, OpenCV, PyTorch, FFmpeg, etc.). The coding agents write generated code to a mounted workspace directory, and the validator agent executes it via a controlled interface that captures output. This design provides three key benefits: namespace isolation protects the host from unsafe code, cgroup limits prevent resource exhaustion, and the containerized environment ensures reproducible execution across deployments.

D. Adaptive Online Monitoring

The final hurdle in the lifecycle of edge video analytics is the non-stationary nature of the deployment environment. Even a perfectly optimized pipeline can suffer from SLO violations due to two primary reasons: (1) Data Drift: changes in video content may affect the workload of vision models [43], [8]. For example, a transition from sparse nighttime traffic to dense daytime congestion increases the computational overhead of post-processing tasks like Non-Maximum Suppression (NMS) and multi-object tracking. (2) Resource Contention: on the edge devices, other background processes triggered by users or thermal throttling can lead to sudden fluctuations in available GPU/CPU cycles [44], [22]. Thus, we propose an *Adaptive Online Monitor* module to make AutoEdge aware of the performance change that violates the initial planning intent after deployment, and enable closed-loop recovery.

Rather than explicitly tracking all potential sources of performance variation, AutoEdge directly monitors the end-to-end latency of deployed pipelines as the primary signal. Specifically, we employ a lightweight sliding-window mechanism that continuously evaluates performance against user-defined SLOs over a configurable window, e.g., 60 frames. To balance responsiveness and stability, model switching is triggered under two conditions: when P_{95} latency exceeds $1.1 \times$ SLO, indicating sustained performance degradation, or when P_{95} latency falls below $0.7 \times$ SLO, indicating opportunity for quality improvement. To ensure predictable latency

and reproducible runtime decisions, AutoEdge applies these deterministic rules locally rather than invoking an LLM, which would introduce decoding delay, cost, and stochastic behavior. When the latency SLO is violated, AutoEdge switches to a more lightweight model variant recommended in the execution plan to restore performance. To minimize switching overhead, the current model continues serving requests while the lightweight variant is being loaded in the background, ensuring uninterrupted pipeline operation. Conversely, when sufficient latency slack is observed, AutoEdge opportunistically upgrades to a more accurate model variant, thereby improving inference quality without compromising SLO compliance.

IV. Implementation

We implement AutoEdge in Python using a server–edge architecture and deploy it on a real-world testbed. The server runs on a MacBook Air (M4) and uses the GPT-4o-mini API as the core LLM for efficiency, while the design allows it to be easily replaced with more advanced models. Edge nodes consist of NVIDIA Jetson Orin Nano and Orin NX devices, each equipped with 8GB of memory (102 GB/s memory bandwidth) and a 1024-core Ampere GPU. To evaluate AutoEdge across diverse and previously unseen hardware conditions, we consider three deployment settings: (1) *Orin Nano*, operating at GPU/CPU frequencies of 0.82/1.27 GHz with default configurations; (2) *Orin NX*, operating at GPU/CPU frequencies of 1.12/1.57 GHz with other configurations unchanged; and (3) *Orin Nano (CPU-only)*, where the GPU is disabled and the CPU runs at 1497 MHz with 6 cores. Communication between the server and edge devices is implemented through a lightweight SSH-based interface, with ngrok [49] used to enable secure remote access and deployment.

Onboarding New Tasks and Models. The current system uses a lightweight, developer-assisted onboarding process to extend its predefined model zoo. For a new query type, the developer first registers its task definition, pipeline dependencies, expected inputs and outputs in the planner and coding-agent prompt templates. Candidate models are then added with their task metadata and profiled latency records on representative hardware configurations, allowing the hardware-aware model selector to consider them. Finally, the corresponding coding agent is instantiated from the existing prompt template, and its generated pipeline is tested by the validator in the edge sandbox before the new capability is made available to users. We discuss automating this onboarding process in §VII-A and leave its implementation to future work.

V. Evaluation

A. Methodology

Queries. We evaluate AutoEdge using the six query types described in Section II-A. We construct a dataset of 600 queries, with 100 queries per type. To ensure both diversity and realism, we manually write 25% of the queries and use GPT-5 to generate the remaining ones by prompting it to follow the same structure and intent as our manually written examples.

Vision Models. Table II summarizes the vision models included in our model zoo. As illustrated in Fig. 2, the number of tasks varies across different queries. To accommodate diverse hardware capabilities, we provide 2–4 representative vision models as candidates for each task. These models span a range of sizes, with accuracy generally increasing with model size. The model zoo allows new models to be easily incorporated.

Model	Task	Used by Query	Params (M)	FLOPs (B)
yolo11n-cls	General Classification (Including vehicles)	Classification	2.8	0.5
yolo11s-cls			6.7	1.6
yolo11m-cls			11.6	4.9
yolo11l-cls			14.1	6.3
yolo11x-cls			29.6	13.6
yolo11n	General Detection (Including vehicles)	Object Detection Counting Tracking License Plate Reco.	2.6	6.5
yolo11s			9.4	21.5
yolo11m			20.1	68
yolo11l			25.3	86.9
yolo11x			56.9	194.9
yolo11n-mot	Motorcyclist Detection	Hel. Vio. Detection	2.6	6.5
yolo11m-mot			20.1	68
yolo11x-mot			56.9	194.9
yolo11n-cls-hel	Helmet Classification	Hel. Vio. Detection	2.8	0.5
yolo11m-cls-hel			11.6	4.9
yolo11x-cls-hel			29.6	13.6
deepsort	Object Association	Tracking	≈3.4	≈0.3
bytesort			NA	NA
yolo11n-plt	License Plate Detection	License Plate Reco.	2.6	6.5
yolo11m-plt			20.1	68
yolo11x-plt			56.9	194.9
easyocr	OCR	License Plate Reco.	≈8-12	≈1-2
fast-plate-ocr			≈1-1.5	≈0.15-0.25

TABLE II: Specifications of existing models in the model zoo. Note that some models are used by multiple query types.

Baselines. As there are currently no fully autonomous systems for edge video analytics deployment, we compare AutoEdge against two categories of closely related baselines. The first category consists of representative LLMs, including GPT-5 [11] and DeepSeek-V3 [13]. The second category includes LLM-based coding agents, specifically Cursor [14] and GitHub Copilot [15], which assist developers in code generation but lack end-to-end development capabilities.

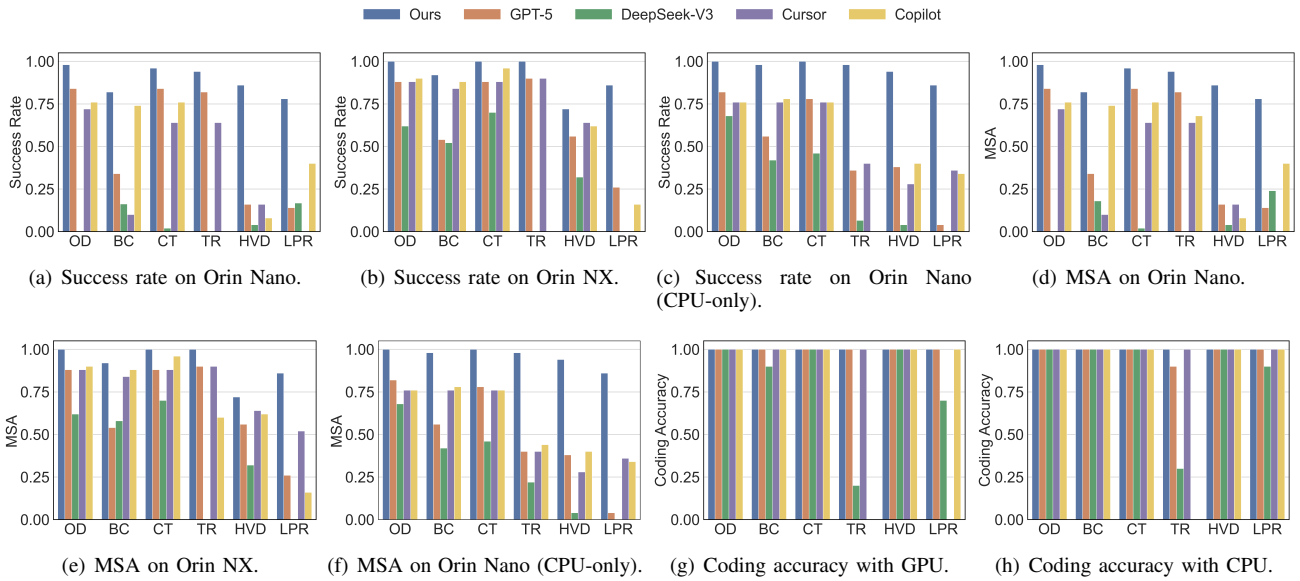
Metrics. We evaluate AutoEdge and all baseline approaches using three metrics: 1) *Model Selection Accuracy* (MSA): evaluates the overall correctness of model selection at each decision step, accounting for both model type and model size. A selection is considered correct only if the end-to-end latency of chosen model(s) meets the latency SLO and achieves the highest accuracy among feasible candidates. Let $m_{q,t}$ denote the model selected at step t for query q, and $\hat{m}_{q,t}$ its ground-truth label. MSA is defined as:

$$MSA = \frac{1}{\sum_q \sum_t} \sum_q \sum_t \{m_{q,t} == \hat{m}_{q,t}\} \qquad (11)$$

2) *Coding Accuracy*: measures the correctness of the generated code across all queries. A query is considered correct if the generated program executes successfully without runtime or logical errors. 3) *Success Rate*: quantifies the probability that the system successfully completes the entire development lifecycle, including model selection, code generation, and deployment. The metric reflects the ratio of queries that are successfully completed to the total number of queries.

B. Overall Performance

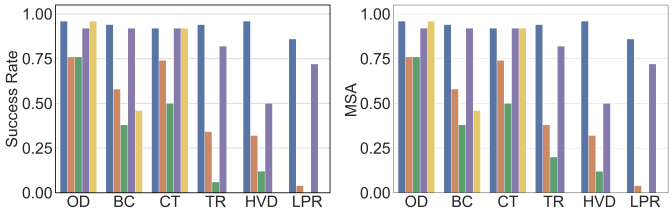
End-to-End Performance. Fig. 13(a), 13(b), and 13(c) report the end-to-end success rate, defined as the joint correctness of model selection and code execution. Across all deployment



(a) Success rate on Orin Nano. (b) Success rate on Orin NX. (c) Success rate on Orin Nano (CPU-only). (d) MSA on Orin Nano.

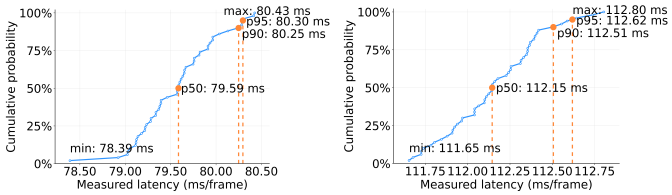
(e) MSA on Orin NX. (f) MSA on Orin Nano (CPU-only). (g) Coding accuracy with GPU. (h) Coding accuracy with CPU.

Fig. 13: Overall performance of AutoEdge and baselines. Measurements are conducted on the NVIDIA Jetson edge devices.



(a) Success rate on Raspberry Pi 5. (b) MSA on Raspberry Pi 5.

Fig. 14: Overall performance of AutoEdge and baselines on Raspberry Pi 5.



(a) Classification (SLO=100ms). (b) Tracking (SLO=120ms).

Fig. 15: End-to-end latency CDFs of representative deployed pipelines.

settings, AutoEdge consistently achieves the superior success rate. On the Orin Nano, AutoEdge attains an average success rate of 89%, outperforming the strongest baseline, GPT-5, by 37%. This trend persists on the Orin NX, where AutoEdge achieves a 92% success rate, exceeding GPT-5 by 25%. Notably, AutoEdge remains robust for complex, multi-stage pipelines, achieving 86% success on license plate recognition (LPR). In contrast, baseline methods exhibit a pronounced *fragility gap*: their performance drops sharply on such queries, with success rates as low as 10% for LPR due to the compounding effects of suboptimal model selection and runtime errors.

We observe similar trends under the CPU-only deployment setting on the Orin Nano in Fig. 13(c). Here, AutoEdge achieves an average success rate of 96%, surpassing the best baseline Cursor by 41%. While baseline methods maintain moderate performance on simpler tasks such as detection and counting, their reliability degrades significantly on multi-stage pipelines. For instance, GPT-5 achieves only 36% success on tracking and 4% on LPR, while DeepSeek fails entirely on both tasks. Cursor and Copilot exhibit similar instability due to incorrect pipeline composition or missing task dependencies. In contrast, AutoEdge maintains consistently high performance across all query types, achieving 98% on tracking and 86% on LPR, demonstrating that its hardware-aware planning and closed-loop validation enable robust end-to-end deployment even under CPU-only constraints.

Fig. 15 reports the measured latency distributions of representative single-stage classification and multi-stage tracking pipelines under SLOs of 100 ms and 120 ms, respectively. Classification achieves a median latency of 79.59 ms and a P_{95} latency of 80.30 ms, while tracking achieves 112.15 ms and 112.62 ms, respectively. The maximum measured latencies remain below their SLOs in both cases, demonstrating stable end-to-end performance.

Model Selection Performance. Next, we evaluate the model selection accuracy (MSA) across six query types on the Jetson Orin Nano and Orin NX platforms. As shown in Fig. 13(d), 13(e), and 13(f), AutoEdge consistently outperforms all baselines, maintaining the highest MSA across all evaluated hardware environments. On the Orin Nano GPU, AutoEdge achieves a mean MSA of 89%, representing a 32% improvement over the strongest baseline, Copilot. The performance gap is particularly notable in complex multi-stage pipelines, with AutoEdge outperforming baselines by an average of 75% for helmet violation detection and 59% for license plate recognition. These trends persist on the Orin NX, confirming that the hardware-aware model selection algorithm in AutoEdge effectively navigates heterogeneous edge resources where standard LLMs struggle with the expanded search space of multi-stage pipelines. Evaluation under CPU-only constraints on the Orin Nano further highlights

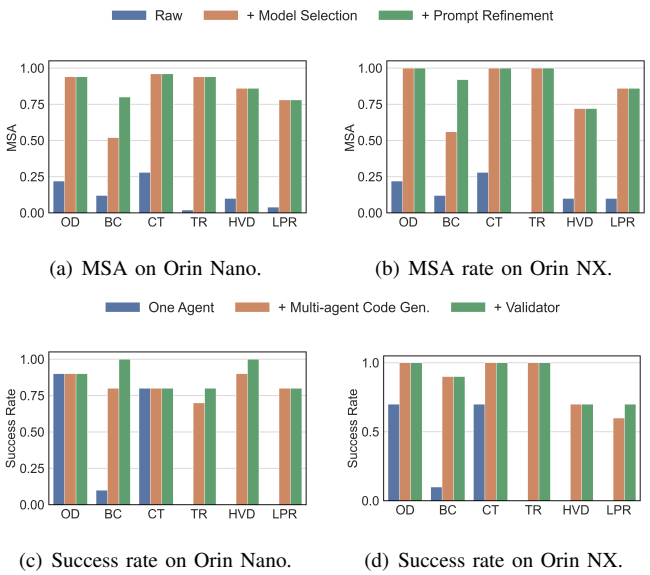
this robustness. AutoEdge achieves a mean MSA of 96%, substantially outperforming Copilot, Cursor, GPT-5, and DS-V3.

Performance on Raspberry Pi 5. To evaluate hardware diversity, we further deploy all methods on a Raspberry Pi 5 [50]. This platform represents a fundamentally different hardware type and configuration from the NVIDIA Jetson devices. It uses a Broadcom BCM2712 with a quad core Arm Cortex A76 CPU and has no NVIDIA CUDA GPU, whereas the Jetson Orin platforms use NVIDIA SoCs with Cortex A78AE CPUs and integrated Ampere GPUs. As shown in Fig. 14, AutoEdge achieves an average MSA of 93.0%, outperforming the strongest baseline, Cursor, by 13 percentage points. AutoEdge also achieves an average success rate of 93.0%, exceeding Cursor by 15.7 percentage points. The improvement is particularly clear for complex pipelines. For tracking, helmet violation detection, and LPR, AutoEdge achieves success rates of 94%, 96%, and 86%, respectively, compared with 82%, 50%, and 56% for Cursor. These results demonstrate that AutoEdge remains effective on a distinct CPU-based edge platform and does not depend on a particular Jetson configuration or CUDA accelerator.

Coding Performance. Fig. 13(g) and 13(h) illustrate the coding accuracy of AutoEdge compared to baseline methods under GPU and CPU-only deployment settings. While all methods perform well on simple single-stage queries such as object detection and counting, AutoEdge is the only system to achieve a perfect 100% accuracy rate across all evaluated query types. This consistency suggests that while off-the-shelf LLMs can handle basic inference code generation, they often lack the compositional logic required for complex multi-stage pipelines. Baseline performance degrades significantly during task orchestration due to architectural pitfalls. Under GPU settings, DeepSeek exhibits substantial accuracy drops in tracking and license plate recognition (LPR) because of interface mismatches and improper device assignment. Cursor fails entirely on LPR as it overlooks the requisite three-stage dependency of placing a license plate detector prior to OCR. Similar trends appear in CPU-only scenarios, where DeepSeek shows a 30% drop in tracking accuracy and Copilot fails the tracking task completely by omitting the preceding detection stage. In contrast, AutoEdge maintains perfect reliability across both hardware configurations. These results highlight the strength of its closed-loop validation, which successfully identifies and resolves dependency errors and task-ordering issues to ensure robust deployment-ready code.

C. Ablation Study

We conduct an ablation study to quantify the contribution of each major component, evaluating two aspects of the pipeline: model selection accuracy (MSA) and end-to-end success rate. All experiments are conducted on the Jetson Orin Nano and Orin NX across six query types. As shown in Fig. 16(a)-16(b), we first compare three progressive configurations: a raw LLM baseline, augmented with our hardware-aware model selection algorithm, and further augmented with prompt refinement. On the Orin Nano, the raw LLM achieves an average MSA of



(a) MSA on Orin Nano. (b) MSA rate on Orin NX.

(c) Success rate on Orin Nano. (d) Success rate on Orin NX.

Fig. 16: Ablation study.

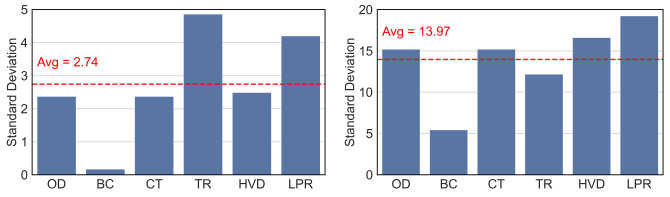
only 13%, with particularly severe failures on multi-stage pipelines—tracking reaches just 0.02 and LPR only 0.04. Adding the model selection algorithm closes most of this gap, increasing tracking to 0.94 and counting to 0.96, as it grounds model selection in the similarity of empirical hardware profiles rather than relying on the LLM's implicit knowledge. Prompt refinement then addresses the remaining reasoning-heavy cases, improving classification from 0.52 to 0.80 while other tasks remain near-perfect. Consistent trends are also observed on the Orin NX. Overall, these results confirm that our model selection algorithm is the dominant contributor to MSA, with prompt refinement providing marginal but important gains on semantically ambiguous tasks.

We also evaluate the impact of the multi-agent code generation and validation design on the end-to-end success rate. The results are shown in Fig. 16(c)-16(d). On the Orin Nano, the single-agent baseline handles simple queries adequately, e.g., detection achieves 0.9, but collapses entirely on complex multi-stage pipelines, with tracking, helmet violation detection, and LPR all reaching 0%. Decomposing code generation into specialized per-task agents substantially recovers this: classification rises from 0.1 to 0.8, tracking from 0 to 0.7, and helmet violation detection from 0 to 0.9. This improvement stems from reduced LLM hallucination and fault isolation, where failures are localized to individual agents rather than contaminating the entire pipeline. Adding the validator further corrects residual workflow errors before execution, pushing classification and helmet violation detection to 1.0 and tracking to 0.8. The same trend holds on the Orin NX. Together, these results show that multi-agent decomposition and pre-execution validation are complementary. The former reduces hallucination and enables precise fault isolation across tasks, while the latter ensures workflow correctness, jointly enabling robust end-to-end deployment.

D. Deep Dive Results

Metric	Before	After
Query-type accuracy	98.0%	99.67%
Task-sequence accuracy	98.0%	99.67%
SLO extraction accuracy	100%	100%
Camera-ID extraction accuracy	82.2%	100%
JSON validity	100%	100%
Missing-field rate	17.8%	0%

TABLE III: Query translation before and after prompt refinement.



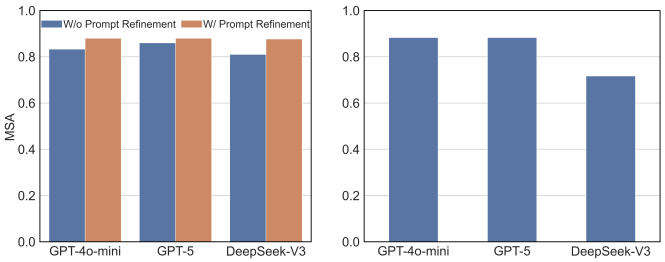
(a) Standard deviation of latency estimation on Orin Nano.

(b) Standard deviation of latency estimation on Orin Nano (CPU-only).

Fig. 17: Latency estimation accuracy of AutoEdge.

Predictor	S1	S2	S3	Avg.
AutoEdge	**2.58**	**35.49**	**3.46**	**13.84**
MLP	6.07	48.47	6.08	20.21
Linear regression	16.49	50.43	8.34	25.09
Random forest	16.49	54.48	15.83	28.93

TABLE IV: MAE of latency prediction methods across three hardware and video settings (ms).



(a) MSA.

(b) Success Rate.

Fig. 18: Performance of AutoEdge across diverse LLMs. Experiments are conducted on Orin Nano.

Query Translation and Prompt Refinement. We evaluate the planner on the 600-query benchmark before and after failure-guided prompt refinement. As shown in Table III, refinement increases query-type accuracy from 98.0% to 99.67% and camera-ID extraction accuracy from 82.2% to 100%, while reducing the missing-field rate from 17.8% to 0%. SLO extraction and JSON validity remain at 100%. Because each query type in our benchmark uniquely determines its task sequence, the LLMs can easily do the mapping. Therefore, task-sequence accuracy is identical to query-type accuracy.

Accuracy of Latency Estimation. Fig. 17(a)-17(b) present the standard deviation of the prediction error across tasks. On Orin Nano with GPU, the estimation error remains consistently low, with standard deviations of 2.37 ms for detection and counting, 0.17 ms for classification, 4.86 ms for tracking, 2.49 ms for helmet violation detection, and 4.20 ms for LPR. These small deviations indicate that the latency predicted by our model selection algorithm closely matches the observed runtime latency across diverse workloads. In contrast, when the GPU is disabled, the variability increases significantly. The standard deviations rise to between 5.44 ms for classification and 19.23 ms for LPR. This increase is expected because CPU-only execution introduces greater runtime fluctuations due to the constrained CPU resources and scheduling effects. Despite this increase, the estimates remain sufficiently accurate to support correct model selection decisions. Overall, the results demonstrate that our latency estimation remains stable and reliable under GPU acceleration, while still maintaining reasonable consistency under CPU-only execution.

Comparison with Alternative Latency Predictors. We compare our latency estimator with a multilayer perceptron (MLP), linear regression, and random forest under three Orin Nano settings: S1 uses six CPU cores and 640p video, S2 uses six CPU cores and 720p video, and S3 uses three CPU cores and 640p video; all settings use GPU and CPU frequencies of 816 and 1267 MHz. As shown in Table IV, our estimator achieves the lowest mean absolute error (MAE) in every setting. Its average MAE is 13.84 ms, compared with 20.21 ms for MLP, 25.09 ms for linear regression, and 28.93 ms for random forest.

Impact of Diverse LLMs. We evaluate the robustness of AutoEdge across different LLM backends, including GPT-4o-mini, GPT-5, and DeepSeek-V3. Figure 18(a) first reports the average MSA when each LLM directly processes queries without additional guidance. In this setting, the models achieve average MSA of 0.833 for GPT-4o-mini, 0.860 for GPT-5, and 0.810 for DeepSeek-V3. When our prompt refinement mechanisms are enabled, performance improves and becomes more consistent across models. The average MSA increases to 0.880 for GPT-4o-mini, 0.880 for GPT-5, and 0.877 for DeepSeek-V3. These results demonstrate that our approach not only improves accuracy but also reduces performance variance across different LLM backends.

We further evaluate end-to-end performance using the full pipeline, including model selection, code generation, and validation. As shown in Fig. 18, both GPT-4o-mini and GPT-5 achieve an average success rate of 0.883, successfully completing most tasks. DeepSeek-V3 attains a lower success rate of 0.717, primarily due to failures in complex multi-stage queries such as LPR. Overall, these results indicate that AutoEdge maintains robust performance across diverse LLM backends, while its system-level design mitigates variability introduced by different models.

Analysis of Validator. Table V summarizes six common error types observed in LLM-generated code, along with the validator's performance in identifying them. As shown in the table, the validator achieves near-perfect accuracy, correctly identifying five out of six error categories with 100% accuracy and achieving 95% accuracy on the remaining category. The only misclassification occurs in the case of device misconfiguration. In this case, the faulty code does not explicitly configure the device, such as moving the model to the GPU. As a result, the validator classifies it as a latency measurement error, since the observed latency may include additional overhead from GPU initialization. This example highlights a subtle ambiguity between configuration errors

Error Type	Accuracy
Incorrect Model Usage	100%
Device Misconfiguration	95%
Latency Measurement Errors	100%
File / Path Errors	100%
Tool Composition Errors	100%
Fake Argument Generation	100%

TABLE V: Performance of the validator in identifying error types of faulty code snippets.

Category	Safety checks	Result
Filesystem isolation	Secret and host file reads; writes to system, model, and video paths; designated workspace write	7/7
Privilege protection	Change non-root identity; Change UID, GID, group, and sudo escalation	8/8
Process containment	Infinite loop; recursive process creation; repeated child-process crashes	3/3
Network isolation	Interfaces; private listener; direct IP, DNS, and payload download attempts	5/5
Cleanup and persistence	Processes; container; network namespace; temporary files; GPU memory	6/6
Total		**29/29**

TABLE VI: Sandbox safety checks and results.

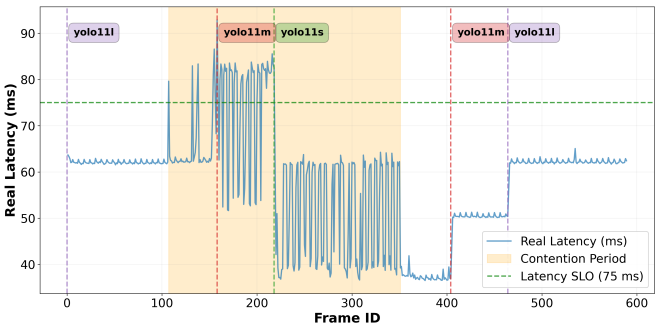


Fig. 19: Dynamic latency monitoring and model switching under GPU contention.

Stage	Time (s)	Token (#)	Price ($)
Query Understanding	3.177	1010	0.000152
Model Selection	0.004		
Plan Generation	5.568		
Multi-Agent Coding	12.589	1020	0.000153
Validation	5.592	734	0.00011
Deploy & Monitor	0.002	0	0
Overall	26.932	2764	0.000415

TABLE VII: Latency and token consumption breakdown.

and their downstream performance effects. Nevertheless, the overall results demonstrate that the validator reliably identifies the majority of code generation failures in AutoEdge.

Sandbox Overhead and Safety. We measure the overhead of creating and removing the edge sandbox. Container creation and startup take 0.158 s and 0.803 s, respectively, for a total startup time of 0.962 s. Removing the container and its workspace takes 0.323 s and 0.154 s, respectively, for a total cleanup time of 0.478 s. When idle, the sandbox uses 2.574 MiB of memory and no measurable CPU resources. Thus, the sandbox introduces only negligible overhead.

We further test filesystem isolation, privilege escalation, process and availability attacks, network isolation, and post-execution cleanup. As shown in Table VI, all 29 safety checks pass. In particular, read-only mounts and non-root execution prevent unauthorized file access and privilege escalation; CPU, memory, process, and execution-time limits restrain infinite loops, recursive process creation, and repeated crashes; and the isolated network namespace blocks external connections and downloads. After each test, forced cleanup removes all descendant processes, container state, network namespaces, temporary files, and GPU-backed unified memory.

Effectiveness of Monitor. We use an example to demonstrate the effectiveness of our monitoring and dynamic model switching mechanism. As shown in Fig. 19, AutoEdge first deploys YOLO11l on the edge device and monitors the P_{95} latency over a sliding window of 60 frames. Initially, latency remains stable at around 62 ms, well below the SLO of 75 ms. At approximately frame 107, we launch a second YOLO11l instance to simulate GPU resource contention. As contention increases, latency spikes become more frequent, causing the P_{95} latency to exceed the upper threshold ($1.1\times$ SLO, i.e., 82.5 ms), which triggers model switching. AutoEdge first downgrades to YOLO11m; when the latency remains above the threshold, it further switches to YOLO11s, where the latency stabilizes below the SLO despite ongoing contention. Around frame 351, the contention is terminated, releasing GPU resources. The P_{95} latency then drops below the lower threshold ($0.7\times$ SLO), prompting the system to scale up the model to improve inference quality without violating SLO. Quantitatively, the first downgrade from YOLO11l to YOLO11m occurs at approximately frame 158, 51 frames after contention begins. Because latency remains above the threshold, AutoEdge switches to YOLO11s at approximately frame 218; the measured latency then remains below the 75 ms SLO for the rest of the contention period. After contention ends at frame 351, AutoEdge upgrades to YOLO11m at approximately frame 404 and returns to the original YOLO11l model at approximately frame 464, demonstrating recovery within 113 frames. These results demonstrate that our monitoring mechanism can effectively adapt to dynamic resource contention and recovery, while consistently maintaining the latency SLO.

System Overhead. Table VII summarizes the latency and token consumption of each stage. The end-to-end process requires 26.9 seconds on average. Among all stages, code generation dominates the runtime with 12.6 seconds, followed by plan generation and validation, which each take around 5.6 seconds. In contrast, the model selection stage introduces negligible overhead with only 0.004 seconds, as it relies on a lightweight deterministic algorithm rather than LLM reasoning. In terms of LLM usage, the framework consumes 2,764 tokens in total, corresponding to an estimated cost of $0.000415 per query. The majority of tokens are used during query understanding and code generation, while the validation introduces a smaller additional cost. Overall, these results show that AutoEdge incurs minimal monetary cost and

modest execution latency, making it practical for real-world deployment scenarios.

VI. Related Work

A. Multi-Agent Collaboration.

Existing LLM-based multi-agent systems commonly use three collaboration workflows [31]. In a *sequential workflow*, agents execute an ordered chain in which each agent consumes outputs produced by preceding agents. MetaGPT [51] encodes software-engineering procedures as an assembly line of specialized roles, while ChatDev [52] organizes design, coding, and testing as sequential phases and subtasks. This workflow naturally captures task dependencies, but early errors can propagate through the chain. In a *parallel workflow*, multiple agents independently generate candidate solutions that are subsequently aggregated. Mixture-of-Agents [53], for example, runs multiple agents in each layer and supplies their outputs to agents in the next layer. Parallel collaboration promotes diverse solutions but introduces aggregation and token overhead. In a *loop workflow*, agents repeatedly critique, debate, or revise outputs until a stopping condition is met. AutoGen [54] supports iterative conversations among configurable agents, and multiagent debate [55] exchanges proposed answers and reasoning over multiple rounds. Such loops support error correction, but repeated interactions can increase latency and cost.

Unlike these fixed collaboration patterns, AutoEdge uses a *customized workflow* generated from each query's task sequence and pipeline dependencies. The planner agent sits on the top, interprets the query, and generates an execution plan based on its task sequence. A self-contained, single-stage query activates one coding agent, whereas a multi-stage query activates only the specialized agents required by its execution plan and orders them according to data dependencies. Each agent produces a pipeline component with explicit inputs and outputs. The validator then creates a localized feedback loop around a component that fails logic validation. This customized workflow provides three advantages for edge video analytics. First, it preserves application semantics by aligning agent boundaries and communication interfaces with executable vision tasks. Second, activating only required agents and regenerating only failed components bounds each agent's context and reduces unnecessary token and regeneration overhead. Third, the modular structure localizes failures and allows a new task capability to be added without redesigning the complete workflow.

B. LLMs for Systems Development

The growing reasoning capabilities of LLMs have motivated their integration into complex system-level tasks that traditionally required expert knowledge and manual configuration. In the networking domain, NetLLM [56] adapts a single LLM to solve diverse networking problems, including adaptive bitrate streaming and cluster job scheduling, by fine-tuning it as a general-purpose networking foundation model. Confucius [57] takes a multi-agent approach, decomposing intent-based network management into subtasks handled by specialized LLM agents augmented with retrieval and rule-based validation to ensure the correctness of critical configurations. In the software engineering domain, ChatDev [52] organizes LLM agents into role-based teams that collaborate through a structured chat chain to produce executable software with reduced hallucinations. SWE-agent [58] wraps an LLM with an agent–computer interface that enables it to autonomously browse codebases, execute tests, and patch bugs on real-world GitHub issues. In closest spirit, AutoIOT [59] and EmbedGenius [60] translate natural language descriptions into executable programs for IoT sensor applications.

While these systems demonstrate the promise of LLM-driven automation, none address the full deployment lifecycle of edge video analytics. Systems like NetLLM [56] and Confucius [57] target networking optimizations within already-deployed infrastructure and do not address model selection or code synthesis for DNN pipelines. Software engineering agents such as ChatDev [52] and AutoIOT [59] handle code development without awareness of the tight resource constraints of edge devices and SLO requirements. AutoEdge addresses this gap by integrating hardware-grounded planning, multi-agent code generation with sandbox validation, and adaptive runtime monitoring into a single, cohesive framework purpose-built for edge video analytics deployment.

VII. Limitations and Future Work

A. Augmentation of Agent's Capacity

While AutoEdge currently relies on a predefined model zoo categorized by representative query types, its agentic architecture provides a natural foundation for dynamic capacity expansion. Future iterations of the system could integrate an online search and integration module within the planner agent. When a user issues a query that falls outside the existing query definitions, such as video anomaly detection, the agent could autonomously browse online model hubs (e.g., Hugging Face [61] or ONNX Model Zoo [62]), evaluate candidate architectures against the target device's hardware summary, and trigger an automated download and profiling routine. By possessing the automatic searching and updating ability, AutoEdge could evolve into a self-growing deployment agent that adapts to a wide range of video analytics domains without manual intervention.

B. Incorporation of Other Optimizations.

Current model selection primarily considers the model type and architecture. However, the configuration space for edge video analytics extends beyond the model to include low-level system optimizations. Future versions of the multi-agent code generation phase could be enhanced to consider a broader suite of optimization techniques, such as post-training quantization (e.g., INT8/FP16), TensorRT engine acceleration, and resolution adaptation. By empowering the planner and coding agents to reason about these cross-layer optimizations, for instance, choosing between a larger model with INT8 quantization versus a smaller model at full precision, the system could achieve a more refined Pareto front between accuracy and latency. Integrating these techniques into the design

would allow AutoEdge to further squeeze performance out of resource-constrained edge hardware, ensuring even tighter adherence to user-defined SLOs in complex environments.

VIII. Conclusion

In this paper, we presented AutoEdge, an agentic framework that automates the full lifecycle of edge video analytics deployment. AutoEdge integrates hardware-aware planning, multi-agent code generation, and adaptive online monitoring to translate user intent into SLO-compliant edge pipelines. Evaluations on Jetson devices and Raspberry Pi 5 demonstrate that AutoEdge generalizes across GPU-accelerated and CPU-based platforms and improves the deployment success rate by 41% over existing LLM-based approaches.

References

[1] K. Du, A. Pervaiz, X. Yuan, A. Chowdhery, Q. Zhang, H. Hoffmann, and J. Jiang, "Server-driven video streaming for deep learning inference," in *Proceedings of the ACM SIGCOMM Conference*, 2020.

[2] J. Li, L. Liu, H. Xu, S. Wu, and C. J. Xue, "Cross-camera inference on the constrained edge," in *Proceedings of IEEE INFOCOM*, 2023.

[3] Y. Li, A. Padmanabhan, P. Zhao, Y. Wang, G. H. Xu, and R. Netravali, "Reducto: On-camera filtering for resource-efficient real-time video analytics," in *Proceedings of the ACM SIGCOMM Conference*, 2020.

[4] B. Zhang, X. Jin, S. Ratnasamy, J. Wawrzynek, and E. A. Lee, "Awstream: Adaptive wide-area streaming analytics," in *Proceedings of the ACM SIGCOMM Conference*, 2018.

[5] "Invigilo AI." https://www.invigilo.ai/category/video-analytics, 2025.

[6] "Boosting Factory Safety with AI Video Analytics." https://bcbs.asia.canon/discover/blog/boosting-factory-safety-with-ai-video-analytics/, 2025.

[7] R. Bhardwaj, Z. Xia, G. Ananthanarayanan, J. Jiang, Y. Shu, N. Karianakis, K. Hsieh, P. Bahl, and I. Stoica, "Ekya: Continuous learning of video analytics models on edge compute servers," in *Proceedings of USENIX NSDI*, 2022.

[8] M. Khani, G. Ananthanarayanan, K. Hsieh, J. Jiang, R. Netravali, Y. Shu, M. Alizadeh, and V. Bahl, "{RECL}: Responsive {Resource-Efficient} continuous learning for video analytics," in *Proceedings of USENIX NSDI*, 2023.

[9] Y. Mao, J. Li, J. Wang, H. Xu, T.-W. Kuo, N. Guan, and C. J. Xue, "Easz: An agile transformer-based image compression framework for resource-constrained iots," in *ACM/IEEE Design Automation Conference (DAC)*, 2025.

[10] "Video Analytics Market Size & Share Analysis - Growth Trends & Forecasts (2025 - 2030)," https://www.mordorintelligence.com/industry-reports/video-analytics-market, 2025.

[11] "OpenAI GPT-5." https://openai.com/gpt-5/, 2025.

[12] "Gemini 3." https://gemini.google.com/, 2025.

[13] "DeepSeek V3." https://www.deepseek.com/, 2025.

[14] "Cursor." https://cursor.com/, 2025.

[15] "GitHub Copilot." https://github.com/features/copilot, 2025.

[16] "Devin." https://devin.ai/, 2025.

[17] Z. Bi, Y. Wan, Z. Wang, H. Zhang, B. Guan, F. Lu, Z. Zhang, Y. Sui, H. Jin, and X. Shi, "Iterative refinement of project-level code context for precise code generation with compiler feedback," in *Findings of the Association for Computational Linguistics*. ACL, 2024.

[18] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty, and S. K. Lahiri, "Llm-based test-driven interactive code generation: User study and empirical evaluation," *Transactions on Software Engineering (TSE)*, 2024.

[19] M. Wong, M. Ramanujam, G. Balakrishnan, and R. Netravali, "{MadEye}: Boosting live video analytics accuracy with adaptive camera configurations," in *Proceedings of USENIX NSDI*, 2024.

[20] D. Kang, P. Bailis, and M. Zaharia, "Blazeit: Optimizing declarative aggregation and limit queries for neural network-based video analytics," *Proceedings of the VLDB Endowment*, vol. 13, no. 4.

[21] D. Kang, J. Emmons, F. Abuzaid, P. Bailis, and M. Zaharia, "Noscope: Optimizing neural network queries over video at scale," *Proceedings of the VLDB Endowment*, vol. 10, no. 11, 2017.

[22] A. Padmanabhan, N. Agarwal, A. Iyer, G. Ananthanarayanan, Y. Shu, N. Karianakis, G. H. Xu, and R. Netravali, "Gemel: Model merging for {Memory-Efficient},{Real-Time} video analytics at the edge," in *Proceedings of USENIX NSDI*, 2023.

[23] Y. Lu, A. Chowdhery, and S. Kandula, "Optasia: A relational platform for efficient large-scale video analytics," in *Proceedings of the ACM SoCC*, 2016.

[24] J. Li, Y. H. Cai, L. Liu, Y. Mao, C. J. Xue, and H. Xu, "Accelerating point cloud analytics on resource-constrained edge devices," *Computer Networks*, 2025.

[25] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *Proceedings of NeurIPS*, vol. 30, 2017.

[26] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[27] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. d. L. Casas, L. A. Hendricks, J. Welbl, A. Clark *et al.*, "Training compute-optimal large language models," *arXiv preprint arXiv:2203.15556*, 2022.

[28] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.

[29] A. Liu *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[30] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," *Proceedings of NeurIPS*, 2022.

[31] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," *arXiv preprint arXiv:2402.01680*, 2024.

[32] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, 2024.

[33] M. Ramanujam, Y. Dai, K. Jamieson, and R. Netravali, "Legilimens: Performant video analytics on the system-on-chip edge," *arXiv preprint arXiv:2504.21136*, 2025.

[34] L. Chen, D. Feng, E. Feng, Y. Wang, R. Zhao, Y. Xia, P. Xu, and H. Chen, "Characterizing mobile soc for accelerating heterogeneous llm inference," in *Proceedings of the ACM SIGOPS SOSP*, 2025.

[35] M. M. H. Shuvo, S. K. Islam, J. Cheng, and B. I. Morshed, "Efficient acceleration of deep learning inference on resource-constrained edge devices: A review," *Proceedings of the IEEE*, vol. 111, no. 1, 2022.

[36] J. Li, Y. H. Cai, L. Liu, Y. Mao, C. J. Xue, and H. Xu, "Moby: Empowering 2d models for efficient point cloud analytics on the edge," in *Proceedings of ACM MM*, 2023.

[37] "NVIDIA Jetson Orin Nano Super Developer Kit." https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/nano-super-developer-kit/, 2025.

[38] "NVIDIA Jetson Orin NX Developer Kit." https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/, 2025.

[39] "NVIDIA GeForce RTX 5080." https://www.nvidia.com/en-us/geforce/graphics-cards/50-series/rtx-5080/, 2025.

[40] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, 2025.

[41] "What are AI hallucinations?" https://www.ibm.com/think/topics/ai-hallucinations, 2025.

[42] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM computing surveys*, 2023.

[43] X. Zeng, B. Fang, H. Shen, and M. Zhang, "Distream: scaling live video analytics with workload-adaptive distributed edge intelligence," in *Proceedings of the ACM/IEEE SenSys*, 2020.

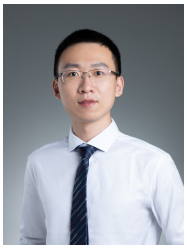
[44] T. Benoit-Cattin, D. Velasco-Montero, and J. Fernández-Berni, "Impact of thermal throttling on long-term visual inference in a cpu-based edge device," *Electronics*, 2020.

[45] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2015.

[46] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE CVPR*, 2016.

[47] "Single-responsibility principle ," https://en.wikipedia.org/wiki/Single-responsibility_principle, 2025.

[48] "NVIDIA JetPack." https://developer.nvidia.com/embedded/jetpack, 2025.
[49] "ngrok," https://ngrok.com/, 2025.
[50] "Raspberry Pi 5," https://www.raspberrypi.com/products/raspberry-pi-5/, 2025.
[51] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin *et al.*, "Metagpt: Meta programming for a multi-agent collaborative framework," in *The twelfth international conference on learning representations*, 2023.
[52] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong *et al.*, "Chatdev: Communicative agents for software development," in *Proceedings of ACL*, 2024.
[53] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou, "Mixture-of-Agents Enhances Large Language Model Capabilities," in *Proceedings of ICLR*, 2025.
[54] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu *et al.*, "Autogen: Enabling next-gen llm applications via multi-agent conversations," in *First conference on language modeling*, 2024.
[55] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving Factuality and Reasoning in Language Models through Multiagent Debate," in *Proceedings of ICML*, 2024.
[56] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, "NetLLM: Adapting large language models for networking," in *Proceedings of the ACM SIGCOMM Conference*, 2024.
[57] Z. Wang, S. Lin, G. Yan, S. Ghorbani, and e. a. Yu, Minlan, "Intent-driven network management with multi-agent llms: The confucius framework," in *Proceedings of the ACM SIGCOMM Conference*, 2025.
[58] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "Swe-agent: Agent-computer interfaces enable automated software engineering," *Proceedings of NeurIPS*, 2024.
[59] L. Shen, Q. Yang, Y. Zheng, and M. Li, "Autoiot: Llm-driven automated natural language programming for aiot applications," in *Proceedings of the ACM MobiCom*, 2025.
[60] H. Yang, M. Li, M. Han, Z. Li, and W. Xu, "Embedgenius: Towards automated software development for generic embedded iot systems," *arXiv preprint arXiv:2412.09058*, 2024.
[61] "Hugging Face," https://huggingface.co/, 2025.
[62] "ONNX Model Zoo," https://github.com/onnx/models, 2025.



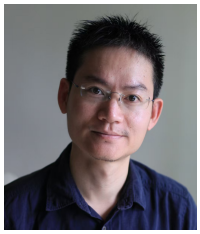
Jingzong Li is currently an Assistant Professor in the Department of Computer Science, The Hang Seng University of Hong Kong. He received his B.Eng. degree from University of Electronic Science and Technology of China in 2019, and his Ph.D. degree in Computer Science from City University of Hong Kong in 2024. His research interests include LLM systems and edge computing. He is a member of ACM and IEEE.



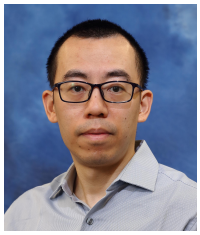
Caoxi Zhang is a Research Assistant in the Department of Computer Science at The Hang Seng University of Hong Kong. She obtained both her Bachelor of Science and Master of Science degrees from Durham University in 2023 and 2024, respectively. Her academic background combines computing and analytical skills, supporting her current research work.



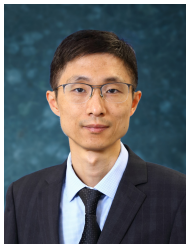
Huanqi Yang received the Ph.D. degree in computer science from the Department of Computer Science, City University of Hong Kong, in 2025, under the supervision of Prof. Weitao Xu. His research interests include mobile agentic systems, generative AI for sensing, and physical layer security. His work has been published in top-tier venues such as ACM SenSys, ACM MobiHoc, IEEE INFOCOM, and IEEE Transactions on Mobile Computing. He is the recipient of the Mark Weiser Best Paper Award at IEEE PerCom 2023 and the Best Paper Award at ACM EIFCom 2024.



Hai Liu received the B.Sc. and M.Sc. degrees in applied mathematics from South China University of Technology and the Ph.D. degree in computer science from the City University of Hong Kong. He is currently a Professor with the Department of Computer Science, The Hang Seng University of Hong Kong (HSUHK). Before joining HSUHK, he held several academic positions at the University of Ottawa and Hong Kong Baptist University. His research interests include wireless networking, cloud computing, artificial intelligence, and algorithm design and analysis. He is a member of IEEE.



Hong Xu is an Associate Professor in Department of Computer Science and Engineering, The Chinese University of Hong Kong. His research area is computer networking and systems, particularly machine learning systems and data center networks. His work has received best paper awards from ACM SIGCOMM 2022, IEEE ICNP 2023 and 2015, among others. He has also received multiple collaborative research awards from industry and distinguished reviewer/editor awards from the community. He is a senior member of ACM and IEEE, and an associate editor of ACM TOCS and IEEE TNSE.



Guoliang Xing (Fellow, IEEE) received the PhD degree from Washington University in St. Louis, in 2006. He is currently a Professor with the Department of Information Engineering, The Chinese University of Hong Kong. He was previously a faculty member with Michigan State University, USA, from 2008 to 2017. He received the US NSF CAREER Award in 2010, the Withrow Distinguished Faculty Award from Michigan State University in 2014, and the Research Excellence Award and the Outstanding Fellow Award from CUHK in 2024. He currently leads several large-scale embedded AI projects on urban smart infrastructure, autonomous driving, and smart health. His work has received six Best Paper Awards, four Best Demo/Poster Awards, and seven Best Paper Finalist honors at premier international conferences including MobiCom, MobiSys, SenSys, ICNP, and IPSN. He is a fellow of IEEE and ACM.